# DRCS ENGINE — MULTI-LLM HANDOFF CAPSULE

**Canonical Project Transfer Package**
Version 1.0

---

## PURPOSE

This capsule transfers the current state of the **Dynamic Response Content System (DRCS) Engine** to another AI system.

The recipient AI should treat this document as a project briefing, operating manual, library index, and continuity package.

**The goal is continuity without re-derivation.**

---

## PROJECT

### Name

**DRCS Engine** — Dynamic Response Content System

### Core Thesis

A content generation and governance engine that operates as eight configurable components (C1–C8) serving as gates. The user provides a natural language prompt; the engine returns a real media file path and caption — either reused from a library or freshly generated.

### Core Principle

**One engine, eight configurable components, deployed via configuration only.** Zero hard-coded tenant logic. Behavior is entirely driven by per-tenant configuration loaded from persistence.

---

## PROJECT STATUS

### Current Phase

**Production Transition** — core engine built, tested, and operational. Natural language entry point live. C1–C6 gates complete. C7–C8 not yet designed.

### Repo

https://github.com/TellDontaMatter/drcs-engine

### Working Directory

`/home/ubuntu/github_repos/drcs-engine`

### Current Branch

`feat/orchestrator-and-ui` (merged into `master` via PR #6)

### Build Status

- ✅ **TypeScript build:** Clean, no errors
- ✅ **Test suite:** 107 tests passing across 10 suites
- ✅ **End-to-end:** Verified live (prompt → LLM → file path + caption returned)
- ✅ **LLM Integration:** Abacus AI routeLLM operational (gpt-4o-mini, `ABACUS_API_KEY` active)
- 🔴 **C7–C8:** Not designed or implemented

---

## LOCKED DESIGN PRINCIPLES

### 1. Single Canonical Codebase

One repo, one build. Deployment-specific behavior achieved **entirely through configuration** — tenant config, category schemas, asset registries. Zero hardcoded tenant IDs in gate logic.

### 2. Multi-Tenant Architecture with Strict Isolation

Every database query, every gate call, every audit trail is tenant-scoped. A tenant's data is invisible to every other tenant. Enforced at persistence layer via `tenant_id` filters on every read/write.

### 3. Fail Closed on Any Log Integrity Issue

C3 (Governor) checks usage-log integrity before allowing deployment. If the log cannot be verified, the gate **fails closed** (blocks deployment).

### 4. C3 Locked Parameters

- `max_count = 3`
- `rolling_window = 30 days`
- `counting_unit = per deployment instance`

Not configurable per request. Overrides require explicit tenant-config policy flags.

### 5. Strict 3-Step Check Before New Generation

C4 (Resolution) **must** attempt these steps in order:
1. **Existing-as-is** — exact caption match
2. **Recaption** — reuse asset, new caption
3. **Escalate to C5** — only if steps 1 & 2 fail

Never generate fresh content if existing content can be reused or recaptioned.

### 6. No Date/Calendar Selection Path

C2 (Situational Bank) has **zero** date/time/weekday logic. Category resolution is driven exclusively by real-time condition signals. Static source scan test confirms no `Date()`, `getDay()`, calendar APIs in compiled C2 output.

**Rationale:** Protected categories like "Friday" are resolved because an upstream signal reports the Friday situation, never because the engine checked the calendar.

### 7. One Gate = One Branch = One PR Targeting Master

Git workflow: every gate built on its own feature branch, merged to `master` via individual PRs. No stacking branches.

**Exception:** Integration PR #6 ( `feat/orchestrator-and-ui` ) combined C3/C4/C5/orchestrator changes because they were interdependent.

---

## COMPLETED WORK

### Gates (C1–C6)

| Gate | Name | Status | Tests | File |
|---|---|---|---|---|
| **C1** | Lock | ✅ Complete | 4 passing | `src/gates/c1/index.ts` |
| **C2** | Situational Bank | ✅ Complete | 6 passing | `src/gates/c2/index.ts` |
| **C3** | Governor | ✅ Complete | 7 passing | `src/gates/c3/index.ts` |
| **C4** | Resolution | ✅ Complete | 12 passing | `src/gates/c4/index.ts` |
| **C5** | Generation | ✅ Complete | 8 passing | `src/gates/c5/index.ts` |
| **C6** | Governance | ✅ Complete | 7 passing | `src/gates/c6/index.ts` |

### Infrastructure

- ✅ **Orchestrator:** `src/orchestrator/index.ts` — runs C6→C2→C4→C5→C3 sequence with short-circuiting + audit trail
- ✅ **Natural Language Entry:** `evaluatePrompt(tenant_id, prompt)` — LLM maps user intent → category name → verdict
- ✅ **Content Retrieval:** `src/assets/index.ts` — maps `asset_id` → `file_path` + `caption`
- ✅ **LLM Integration:** `src/llm/index.ts` — Abacus routeLLM, gpt-4o-mini, error handling
- ✅ **Persistence:** Prisma ORM, 7 models (AssetRegistry, CategorySchema, UsageLog, GovernanceRecord, TenantConfig, GateState, ProposalApprovalLog)
- ✅ **Migrations:** All applied; `file_path` and `caption` columns added to `AssetRegistry`
- ✅ **Seeds:** `seedZilly()` — canonical Zilly tenant data with file paths and captions

### Surfaces

- ✅ **Web Console:** `src/server/index.ts` → http://localhost:3000 — single-page form, prompt input + structured fallback
- ✅ **CLI:** `src/cli/index.ts` → `npm run evaluate --prompt "..."`

- ✅ **Programmatic API:** `src/index.ts` exports `evaluate()`, `evaluatePrompt()`

---

# ERRORS & FIXES (HISTORY)

## Initial Build Error: Gates Were Pure Functions Without Enforcement

**Problem:** Original vision was eight pure gates. It shipped as “gears with no machine” — gates returned decisions but didn’t retrieve content or call the LLM.
**Fix:** Added the **Orchestrator** layer and **C5** (Generation). The orchestrator runs the gate sequence and retrieves content; C5 handles LLM-based caption generation.

## Merge Conflicts During Integration PR #6

**Problem:** README.md and src/index.ts had conflicts when combining C3/C4/orchestrator branches.
**Fix:** Manual cleanup, rewriting the intro, consolidating exports.

## LLM Prompt Mismatch for Category Mapping

**Problem:** Original `extractPromptFields` let the LLM generate free-form “situation” text (e.g., “Celebrating 10k followers”). C2 does exact name matching against category names (e.g., “Victory”). The two never connected.
**Fix:** Redesigned `extractPromptFields` to:
1. Load the tenant’s actual category names from DB
2. Pass that closed list to the LLM
3. Ask LLM to pick the single best-matching category name verbatim
4. Return that exact name so C2 can match it

Now: prompt → LLM → real category name → C2 resolves it → content returned.

## Missing File Paths in Asset Registry

**Problem:** Seeded assets had `asset_id` and `caption` but `file_path` was null. Engine returned asset IDs instead of real media paths.
**Fix:** Updated `seedZilly()` to populate `file_path` for all assets. Added `updateFilePath()` function to `assetRegistry` repository.

---

# PENDING TASKS & NEXT STEPS

## Immediate (Within Current Scope)

1. **Test multi-tenant isolation in production** — verify no cross-tenant data leakage
2. **Seed real media files for all Zilly categories** — currently only 3 of 8 have real file paths
3. **Monitor C5 `ESCALATE_FAILED` rate** — track LLM errors in production
4. **Deploy to production PostgreSQL** — currently running on SQLite (dev.db)

## Future (Beyond Current Build)

1. **Design & Implement C7 (Compliance Gate)** — no spec exists yet
2. **Design & Implement C8 (Distribution Gate)** — no spec exists yet
3. **Add second tenant** — verify zero code changes required for new tenant

4. **Optimize LLM prompt** — currently 2-step (prompt extraction + C5 generation); explore single-call optimization
5. **Add file upload surface** — currently assumes assets pre-seeded; add "upload new asset" flow

---

# STANDING CONSTRAINTS & PREFERENCES

## Database

- **Dev/Test:** SQLite ( `prisma/dev.db` , `prisma/test.db` )
- **Production:** PostgreSQL (connection string in `DATABASE_URL` env var)
- **ORM:** Prisma (schema at `prisma/schema.prisma` )

## LLM API

- **Endpoint:** Abacus AI routeLLM ( `https://routellm.abacus.ai/v1/chat/completions` )
- **Model:** `gpt-4o-mini`
- **Auth:** `ABACUS_API_KEY` environment variable
- **Temperature:** 0.4 (C5), 0.2 (prompt extraction)

## Git Workflow

- **Branch strategy:** One gate per branch, each PR targets `master` directly
- **No stacking branches** (exception: integration PR when gates are interdependent)
- **Commits:** Semantic messages (e.g., `feat: add C5 generation gate` , `fix: map prompt to category name` )

## Testing

- **Framework:** Jest
- **LLM calls:** Mocked in all tests ( `jest.spyOn(llm, 'callLLM')` )
- **Database:** In-memory SQLite ( `test.db` ) reset before each suite
- **Tenant isolation:** Every test suite includes multi-tenant verification

## Code Quality

- **TypeScript:** Strict mode enabled
- **Linting:** None configured yet (add ESLint/Prettier if desired)
- **Build command:** `npm run build` (outputs to `dist/` )
- **Test command:** `npm test` (runs all suites)

---

# ENVIRONMENT & KEY FACTS

## Secrets / Credentials

- **GitHub token:** stored under secret name `githubuser`
- **Abacus API key:** `ABACUS_API_KEY` environment variable (live in this session)

## Tenant Data

- **Reference tenant:** `zilly` — fitness cardio content (8 canonical clips: gentle start → victory)
- **Demo seed:** `src/seeds/demo.ts` — illustrative only, NOT canonical

- **Real seed:** `src/seeds/zilly.ts` — canonical Zilly data with file paths and captions

### File Structure

```
drcs-engine/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   ├── dev.db (SQLite dev database)
│   └── test.db (SQLite test database)
├── src/
│   ├── gates/
│   │   ├── c1/ c2/ c3/ c4/ c5/ c6/
│   ├── orchestrator/
│   ├── assets/
│   ├── llm/
│   ├── persistence/
│   ├── seeds/
│   ├── server/
│   ├── cli/
│   ├── types/
│   └── index.ts (public API entry point)
├── tests/
│   ├── c1.test.ts ... c6.test.ts
│   ├── orchestrator.test.ts
│   ├── assets.test.ts
│   ├── persistence.test.ts
│   └── media.test.ts
├── media/
│   └── assets/
│       └── zilly_mascot.png
├── package.json
├── tsconfig.json
├── jest.config.js
└── README.md
```

---

## DECISIONS & REJECTED ALTERNATIVES

### Decision: Add Orchestrator Layer

**Problem:** Gates alone were “gears with no machine.”
**Solution:** Built orchestrator to run gate sequence, handle short-circuiting, retrieve content.

### Decision: Add C5 (Misalignment Protocol)

**Problem:** C4 escalation had nowhere to go.
**Solution:** Built C5 to handle LLM-based fresh caption generation.

### Decision: Add Natural Language `evaluatePrompt` Entry Point

**Problem:** Multi-field form unusable for intended user (“make content out of any idea”).
**Solution:** Added `evaluatePrompt(tenant_id, prompt)` — one text input, LLM maps intent to category.

### Decision: LLM Prompt Redesign for Category Mapping

**Problem:** LLM-generated "situation" text never matched category names.
**Solution:** Load tenant's category names from DB, pass to LLM as closed list, ask LLM to pick one verbatim.

### Rejected: Hardcoded Tenant Logic in Gates

**Why:** Violates single-codebase principle.
**Chosen:** Tenant-scoped persistence; gates load config/categories/assets at runtime.

### Rejected: Seed Demo Data as Canonical

**Why:** `demo` seed is illustrative only, not the real Zilly mapping.
**Chosen:** Treat `demo` as non-canonical; real Zilly taxonomy in `zilly` seed.

---

## CURRENT OBJECTIVE

**Do not** redesign the architecture. **Do not** rewrite gates. **Do not** change locked design principles unless explicitly instructed.

**Priority (execution):**
1. ✅ **Publish existing engine** — done (web console + CLI + programmatic API all live)
2. 🔄 **Seed real media files** — 5 of 8 Zilly categories still have `file_path: null`
3. 🔴 **Deploy to production** — migrate from SQLite to PostgreSQL
4. 🔴 **Design C7–C8** — no spec exists yet
5. 🔴 **Add second tenant** — verify zero code changes required

---

## INSTRUCTION TO RECIPIENT AI

**Assume:**
- Design principles are locked
- C1–C6 gates are locked
- Orchestrator is locked
- Natural language entry is locked
- Persistence schema is locked

**Your responsibility is to:**
- **Execute** on the pending tasks (seed files, production deploy, C7–C8 design)
- **Maintain** the test suite (keep all 107 tests passing)
- **Preserve** tenant isolation and fail-closed guarantees
- **Extend** the engine only by adding new tenants or new gates (C7–C8)

**Do not restart discovery unless explicitly instructed.**

**Land the plane.**

---

## PROJECT INSIGHT

The DRCS engine is not a chatbot. It is not a CMS. It is a **governance and generation pipeline** that sits between human intent and deployed content, enforcing policy, reusing existing content, and generating fresh content only when necessary — all while maintaining a complete audit trail of every decision.

**The silence (when it works) is the user typing one sentence and getting back exactly the media file they need, with zero visible machinery.**

---

## HANDOFF COMPLETE

**Recipient AI:** You now have full context. The engine is built, tested, and operational. Your next move is listed under “CURRENT OBJECTIVE.” Proceed.

**Project owner:** This capsule + the documentation library ( `/home/ubuntu/drcs-docs/` ) + the codebase ( `/home/ubuntu/github_repos/drcs-engine/` ) constitute the complete transfer package.